

JUMP GAME II

You are given:

```
nums = [2, 3, 1, 1, 4]
```

You start at index 0.

This time, instead of asking:

Can we reach the last index?

we ask:

What is the minimum number of jumps required to reach the last index?

Approach 1 (Dynamic Programming)

```

class Solution {
public:
    int jump(vector<int>& nums) {
        int n=nums.size();

        // dp[i] = minimum jumps required
        // to reach index i
        vector<int> dp(n,INT_MAX);

        dp[0]=0;

        for(int i=0;i<n;i++){

            // Agar i tak pahuch hi nahi sakte
            // to isko skip kar do
            if(dp[i]==INT_MAX){
                continue;
            }

            // i se jitne jumps le sakte hain
            for(int j=1;j<=nums[i];j++){

                if(i+j<n){

                    // i+j tak pahuchne ke liye
                    // ek aur jump lagega
                    dp[i+j]=min(dp[i+j],dp[i]+1);

                }

            }

        }

        return dp[n-1];
    }
};

```

Complexity

Time -> $O(n^2)$

Space -> $O(n)$

Approach 2 (Optimized Greedy)

```
class Solution {
public:
    int jump(vector<int>& nums) {
        int n=nums.size();

        int jumps=0;
        int currrange=0;
        int nextrange=0;

        int i=0;
        while(i<n-1){

            // Current range ke andar ke saare indexes dekhenge
            while(i<=currrange){
                nextrange=max(nextrange,i+nums[i]);
                i++;
            }
            // Current range completely process ho gayi
            // ab next range mein jaane ke liye
            // ek jump lena padega
            jumps++;
            currrange=nextrange;
        }

        return jumps;
    }
};
```